MSBA 402 – Fall 2026

Intro to SQL and Basic Data Management

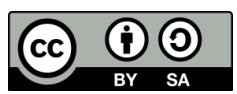
Lecture 3

October 19, 2026

Prof. Ryan Rosario

Anderson School of Management – Business Analytics

University of California, Los Angeles



This work is published under a Creative Commons BY-SA license.

A Timely Tweet

This tweet (only relevant portion is shown) gained a lot of attention from software engineers and data system professionals and is related to what we have been discussing.



Question: What are some reasons why SSN may be duplicated?

Introduction

If we have time, we will do a demo, otherwise it will be part of the practice problem set.



Introduction

In it we will also do some data management tasks.

These require you to use your **intuition** and **analytical skills**.

Introduction

'Follow your heart
Your intuition
It will lead you in the right direction
~~Let go of your mind~~
Your intuition
It's easy to find
Just follow your heart, baby.'

- *Intuition* by Jewel, 2003



Introduction

‘ *La la la la, la la la la, la la la la*
La la’

- *Intuition* by Jewel, 2003



Outline

- Fetching from a Single Table
- Control Flow
- Aggregating Rows
  - Grouping Rows and Computing Aggregates
  - Filtering on Aggregates
- SQL Rules for Quizzes/Exams

The SELECT Statement

- We can write queries against a single relation, or against multiple relations.
- Results from standard SELECT statements are **ephemeral** unless we write the results to a file or write them to a new table.
- **Nothing that occurs only in a SELECT statement changes the data in the table!**

A Basic SQL SELECT Template

We will start by querying a single relation.

```
SELECT
  col1,
  col2 AS new_col2,
  f(col3) AS new_col3,
  ...
  coln
FROM tablename
WHERE <predicate>
-- ORDER BY and WHERE are optional
ORDER BY column_or_set_of_columns;
```

Functions, Arithmetic, and Aliasing

We can apply transformation functions, perform arithmetic on columns and rename (or **alias**) columns to make the output clearer. Recall our `youtube_video` table.

```
SELECT
  video_id,
  title,
  (views / 1000000.0) AS millions_views,
  CAST(likes * 100.0 / NULLIF(likes + dislikes, 0)
    AS DECIMAL(5,2)) AS pct_liked
FROM youtube_video;
```

Transformation functions apply to **each row independently in the table.**

Functions, Arithmetic, and Aliasing

Transformation Functions (Independent Rows)

Let $f(\texttt{price}) = 1.1 * \texttt{price}$

FROM products

product	price
apple	1.50
banana	0.80
carrot	1.20
lettuce	2.00

f**(price)**

product	f(price)
apple	$f(1.50)$
banana	$f(0.80)$
carrot	$f(1.20)$
lettuce	$f(2.00)$

SELECT product, 1.1 * price as price_with_tax, category;

product	price_with_tax
apple	1.65
banana	0.88
carrot	1.32
lettuce	2.20

Functions, Arithmetic, and Aliasing

On the other hand...

Aggregation Functions[1] (Grouped/Dependent Rows)

Let $f_{\texttt{category}}(\texttt{price}) = \texttt{sum(price)}$ by $\texttt{category}$

product	$f_{\texttt{category}}\,(\texttt{price})$	**category**
apple	$f\begin{pmatrix}1.50\\0.80\end{pmatrix}$	fruit
banana		
carrot	$f\begin{pmatrix}1.20\\2.00\end{pmatrix}$	vegetable
lettuce		

We will go into more details in a bit.

[1] Note that a global aggregate considers each row to be its own group

The WHERE Clause

WHERE...:

- uses our usual binary operators
- we can connect predicates with AND, and OR.
- we can also test set containment with IN (set of values) and NOT IN (set of values), where the set is usually computed in subquery
- We can also test for IS NULL or IS NOT NULL

=, >, <, >=, <=, !=

(Some databases and database users use <> instead of !=.)

The `WHERE` Clause

The `WHERE` clause pre-filters rows in a table. It acts on

1. values in columns
2. transformation functions applied on rows, independently (**typically not aggregation functions**)

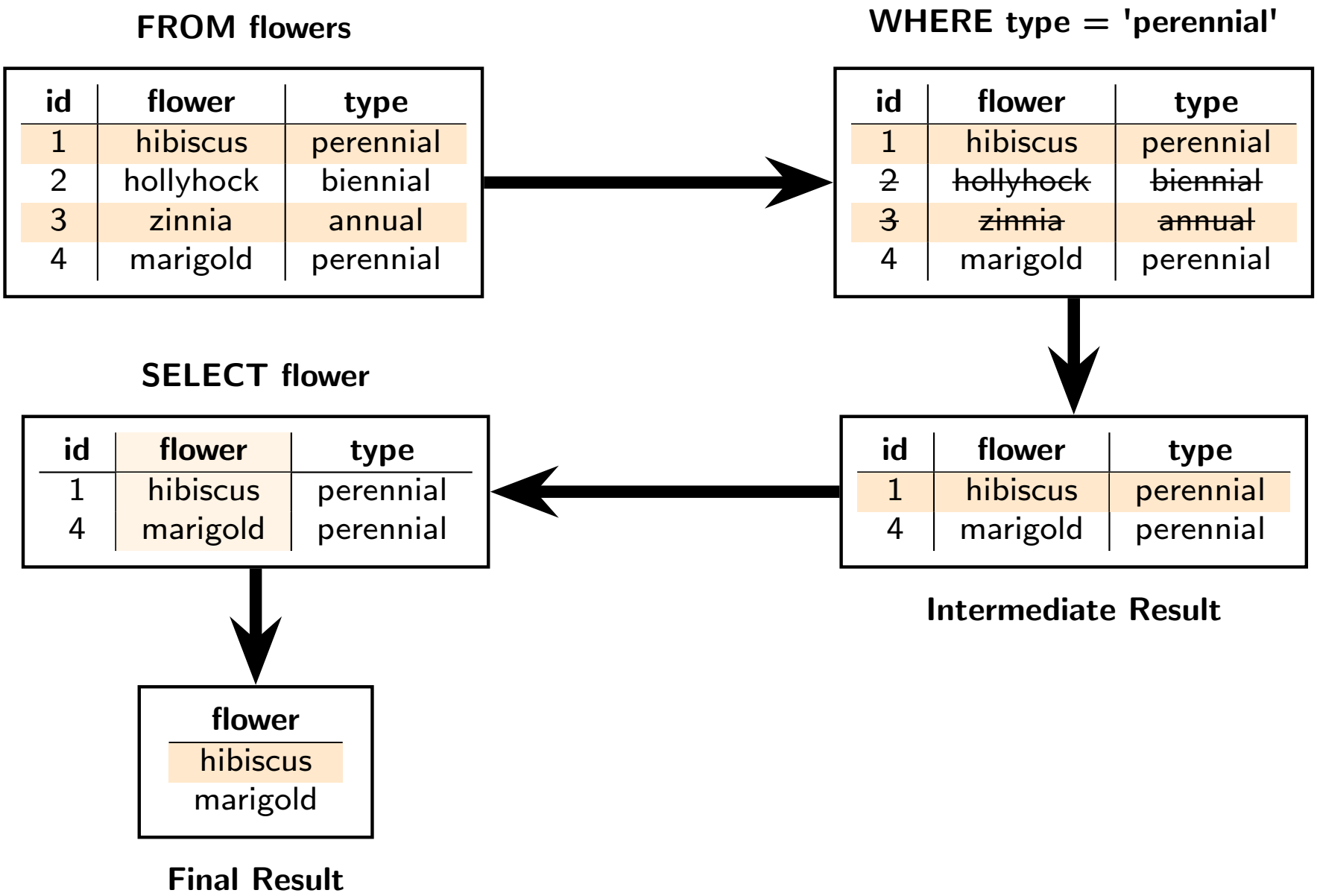
SELECT vs. WHERE

SELECT and WHERE are best friends, but SELECT can live without WHERE when necessary. How sad!

Remember:

1. SELECT extracts **columns** from table.
2. WHERE filters out **rows**, and does so before SELECT

SELECT vs. WHERE



Removing Duplicates with `DISTINCT`

We can remove duplicates with the construct `SELECT DISTINCT`.

Note that `DISTINCT` is distributed across **all** columns in the `SELECT` statement.

```
SELECT DISTINCT title FROM movies;
```

movie_id	title	year
101	Alice in Wonderland	1951
102	Willy Wonka and the Chocolate Factory	1971
103	~~Alice in Wonderland~~	2010
104	Charlie and the Chocolate Factory	2005

Ordering Rows with `ORDER BY`

Often times we want to order the data according to some attribute, or set of attributes.

Let's sort `tinybase`[2] on gpa in descending order. If there are ties[3], sort on `lastname` in ascending order. If still ties, sort on `firstname` ascending.

`lastname`	`firstname`	`mi`	gpa
SCHMOE	JOE	NULL	2.0
BRUIN	JOSIE	NULL	3.8
BRUIN	JOE	NULL	3.85
FRANCO	JAMES	NULL	3.5
BEAR	TED	E.	3.8
COYOTE	WILE	E.	NULL[4]
BLOCK	GENE	D.	4.0

[2] `tinybase` is a simplified `bruinbase` made up for teaching: just names and GPAs, with a tie and a `NULL` to sort around.

[3] a tie means we have two identical values for an attribute in the column being sorted

[4] We have made Wile E. Coyote a Freshman.

Ordering Rows with ORDER BY

We use ORDER BY. We can also use LIMIT to restrict the number of rows returned.

```
SELECT
  lastname AS family_name,
  firstname AS given_name,
  gpa AS grade_point_average
FROM tinybase
WHERE gpa IS NOT NULL
ORDER BY gpa DESC, lastname, firstname
LIMIT 3;
```

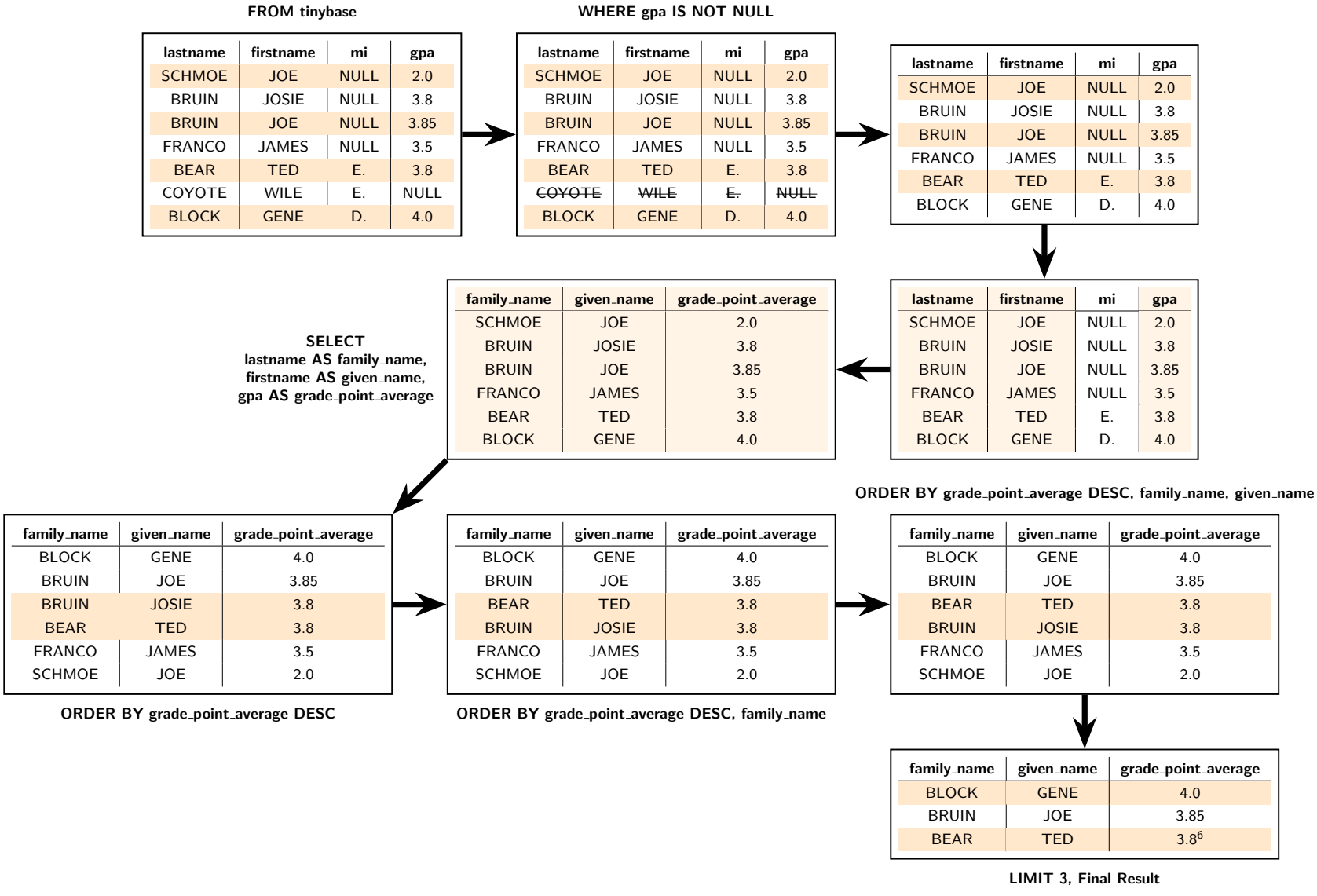
The default is to sort in **ascending** order...[5]

Rule for Exam: Only use column names in ORDER BY, not column sequence numbers.

---

[5] PostgreSQL sorts NULL above every value, so in DESC order Wile E. Coyote would top this list without the WHERE. NULLS LAST puts NULLs at the end.

Ordering Rows with ORDER BY



lastname	firstname	mi	gpa
SCHMOE	JOE	NULL	2.0
BRUIN	JOSIE	NULL	3.8
BRUIN	JOE	NULL	3.85
FRANCO	JAMES	NULL	3.5
BEAR	TED	E.	3.8
COYOTE	WILE	E.	NULL
BLOCK	GENE	D.	4.0

lastname	firstname	mi	gpa
SCHMOE	JOE	NULL	2.0
BRUIN	JOSIE	NULL	3.8
BRUIN	JOE	NULL	3.85
FRANCO	JAMES	NULL	3.5
BEAR	TED	E.	3.8
~~COYOTE~~	~~WILE~~	~~E.~~	~~NULL~~
BLOCK	GENE	D.	4.0

lastname	firstname	mi	gpa
SCHMOE	JOE	NULL	2.0
BRUIN	JOSIE	NULL	3.8
BRUIN	JOE	NULL	3.85
FRANCO	JAMES	NULL	3.5
BEAR	TED	E.	3.8
BLOCK	GENE	D.	4.0

family_name	given_name	grade_point_average
SCHMOE	JOE	2.0
BRUIN	JOSIE	3.8
BRUIN	JOE	3.85
FRANCO	JAMES	3.5
BEAR	TED	3.8
BLOCK	GENE	4.0

lastname	firstname	mi	gpa
SCHMOE	JOE	NULL	2.0
BRUIN	JOSIE	NULL	3.8
BRUIN	JOE	NULL	3.85
FRANCO	JAMES	NULL	3.5
BEAR	TED	E.	3.8
BLOCK	GENE	D.	4.0

family_name	given_name	grade_point_average
BLOCK	GENE	4.0
BRUIN	JOE	3.85
BRUIN	JOSIE	3.8
BEAR	TED	3.8
FRANCO	JAMES	3.5
SCHMOE	JOE	2.0

family_name	given_name	grade_point_average
BLOCK	GENE	4.0
BRUIN	JOE	3.85
BEAR	TED	3.8
BRUIN	JOSIE	3.8
FRANCO	JAMES	3.5
SCHMOE	JOE	2.0

family_name	given_name	grade_point_average
BLOCK	GENE	4.0
BRUIN	JOE	3.85
BEAR	TED	3.8
BRUIN	JOSIE	3.8
FRANCO	JAMES	3.5
SCHMOE	JOE	2.0

family_name	given_name	grade_point_average
BLOCK	GENE	4.0
BRUIN	JOE	3.85
BEAR	TED	3.8[6]

6 Even though Josie had the same GPA, she doesn't appear due to the second column in ORDER BY. Sad.

`SELECT *`

Do not use it!

- schema drift
- performance (especially distributed)
- overfetch
- ambiguous columns after joins

Column Aliasing and Execution Order

The logical order of execution is as follows (from first to last):

1. `FROM` — choose and join tables
2. `WHERE` — filter rows
3. ...
4. `SELECT` — choose columns or expressions
5. `DISTINCT` — remove duplicates
6. `ORDER BY` — sort the results
7. `LIMIT` — return a subset of rows

- **Column aliases** are created in the `SELECT` stage and are only available after it — so they can be used in `ORDER BY`, but not in `WHERE` or `GROUP BY`.
- **Different RDBMS handle aliases differently. We use the ANSI SQL Standard.**
- **You should plan your queries according to this order.**
- We will add more stages to the execution in a bit.

Query Authoring Order

The order of execution is very different from the **order in which you write the statements** which is currently as follows:

```
SELECT
...
FROM
WHERE
...
...
ORDER BY
LIMIT
OFFSET
```

Tip: It is easier to *plan* your queries according to order of execution.

Rules for Quizzes/Exams

We have reached our first few rules for quizzes and exams –

1. Never use `SELECT *`. Write out the columns you want to fetch.
2. In `ORDER BY`, use column names, not numbers.

Always explicitly list out columns. This will be a pattern you will see in my rules.

Outline

- Fetching from a Single Table
- Control Flow
- Aggregating Rows
  - Grouping Rows and Computing Aggregates
  - Filtering on Aggregates
- SQL Rules for Quizzes/Exams

Control Flow

There will be times when we want to change table column output based on the value that is returned from the query.

1. A `CASE` operator that works similar to a `switch`, or a series of `if / else if / else` statements in other languages;
2. A `NULLIF()` construct that returns `NULL` on invalid data, based on an equality.

Just in CASE

Use CASE as a SQL standard version of IF[7].

- CASE is used for condition logic inside a query.
- There are two versions of CASE: simple and searched.
- CASE is often used to **pivot** data (later).
- It is used as a column in SELECT and other constructs.

For illustration purposes we will return to bruinbase.

[7] PostgreSQL has no IF() function in queries. See conditional expressions.

CASE Miscellany

You may be wondering why we need to explicitly give a name to the resulting column with `END AS`.

If we don't, PostgreSQL usually names the column `case`. With both of our `CASE` columns in one query, we get

```
   case    |      case
-----------+----------------
 Freshman  | Undergraduate
```

Two columns with the same useless name. Total rubbish!

CASE Miscellany

According to *Learning SQL* by Beaulieu, the CASE statement can appear elsewhere in a query and other constructs we have discussed can appear within a CASE. I personally find these use cases to be quite esoteric so we will end the discussion of CASE here.

1. In an ORDER BY to create an ad hoc column to order the result by, without actually returning the column.
2. Subqueries can appear in CASE, WHEN, THEN and ELSE to simulate a join.
3. EXISTS can be used in WHEN.
4. It can also be used to simulate the OLAP Pivot operation, and we will see this later.

Simple CASE

Similar to `switch` in other languages. We specify the column and then specify the possible values it can take and what happens when we see that value.

```
SELECT
  ...,
  CASE level
    WHEN 'UFR' THEN 'Freshman'
    WHEN 'USO' THEN 'Sophomore'
    WHEN 'UJR' THEN 'Junior'
    WHEN 'USR' THEN 'Senior'
    WHEN 'GMT' THEN 'Masters Student'
    WHEN 'GD1' THEN 'Doctoral Student'
    WHEN 'GD2' THEN 'Doctoral Candidate'
    WHEN 'CHN' THEN 'Chancellor Bossman'
    ELSE 'Unclassified'
  END AS new_level
  ...
FROM bruinbase;
```

Searched CASE

The **searched** CASE is much more powerful. We can write arbitrary conditional functions based on one or more columns using whichever comparators we need.

```
SELECT
  ...,
  CASE
    WHEN level IN ('UFR', 'USO', 'UJR', 'USR') THEN 'Undergraduate'
    WHEN level IN ('GMT', 'GD1', 'GD2') THEN 'Graduate'
    WHEN level = 'CHN' THEN 'Chancellor'
    ELSE 'Unknown'
  END AS new_career
  ...
FROM bruinbase;
```

Outline

- Fetching from a Single Table
- Control Flow
- Aggregating Rows
  - Grouping Rows and Computing Aggregates
  - Filtering on Aggregates
- SQL Rules for Quizzes/Exams

This Section

- Fetching from a Single Table
- Control Flow
- Aggregating Rows
  - Grouping Rows and Computing Aggregates
  - Filtering on Aggregates
- SQL Rules for Quizzes/Exams

Introduction to Aggregation

Unlike software engineers, business analysts spend considerable time analyzing and summarizing **groups** of data points / rows, not individual[8] rows.

We can summarize the values in a particular column or set of columns using **aggregation functions**.

This is really important for producing reports and dashboards.

[8] It can be important though for studying outliers

Aggregation Functions

Category	Functions
Counting	COUNT(*), COUNT(column_name) COUNT(DISTINCT column_name)
Basic Statistics	SUM, AVG, MAX, MIN
Standard Deviation[9]	STDDEV_SAMP() STDDEV_POP()
Variance	VAR_SAMP(), VAR_POP()
Covariance[10]	COVAR_POP()
Concatenation	STRING_AGG(column_name, delimiter)
Misc	ANY_VALUE()[11]

[9] Avoid using `STDDEV` as it varies, in terms of sample and population, depending on RDBMS.

[10] Same issue as with `STDDEV`.

[11] New in PostgreSQL 16; returns an arbitrary value from the group.

Basic (Global) Aggregations

Unlike a transformation function, an **aggregation function** executes across **groups** of rows. For now, we can compute global aggregates:

```
SELECT
  MIN(gpa)    AS lowest_gpa,              -- NULL values ignored
  MAX(gpa)    AS highest_gpa,             -- NULL values ignored
  AVG(gpa)    AS average_gpa,             -- NULL values ignored
  COUNT(gpa)  AS students_with_grades,    -- counts only non-NULL
  COUNT(*)    AS students                 -- counts all rows
FROM tinybase;
```

Aggregation with Groups

While we can compute global aggregates, it is not very powerful. Instead, we usually want to compute aggregates based on some **group**.

The group can consist of a **single** column or **multiple** columns.

We use `GROUP BY` to form the groups for aggregation.

Aggregation with Groups

For example, to compute the average GPA by major:

```
SELECT
    major,
    AVG(gpa) AS average
FROM bruinbase
GROUP BY major;
```

Important note: In `SELECT`, the grouping column(s) is specified first, followed by the aggregation!

Aggregation with Groups

FROM bruinbase

id	major	gpa
1	Math	3.8
2	Stats	3.2
3	Stats	3.0
4	Math	3.7
5	Comp Sci	2.8

GROUP BY major

id	major	gpa
5	Comp Sci	2.8
1	Math	3.8
4	Math	3.7
2	Stats	3.2
3	Stats	3.0

AVG(gpa)... GROUP BY major

id	major	AVG(gpa)
5	Comp Sci	AVG(2.8)
1 4	Math	AVG $\begin{pmatrix} 3.8 \\ 3.7 \end{pmatrix}$
2 3	Stats	AVG $\begin{pmatrix} 3.2 \\ 3.0 \end{pmatrix}$

SELECT major, AVG(gpa) AS average...

major	average
Comp Sci	2.8
Math	3.75
Stats	3.1

Exercises

Question. What if we only want to look at specific majors?

Question. What if we want average GPA by both major and class level?

Common Error with Aggregation and GROUP BY

What happens here?

```
SELECT
  id,
  major,
  AVG(gpa) AS average
FROM bruinbase
GROUP BY major;
```

id	major	AVG(gpa)
5	Comp Sci	AVG(2.8)
1 4	Math	AVG$\begin{pmatrix} 3.8 \\ 3.7 \end{pmatrix}$
2 3	Stats	AVG$\begin{pmatrix} 3.2 \\ 3.0 \end{pmatrix}$

Problem. Which `id` do we pick? Or do we?

Common Error with Aggregation and `GROUP BY`

Important!

When we use an aggregation function (e.g. `SUM`, `AVG`) with a `GROUP BY`, **every** column in the `SELECT` must either be:

1. called by an aggregation function; or
2. be part of the `GROUP BY`

Common Error with Aggregation and `GROUP BY`

Our previous query would yield:

```
ERROR:  column "bruinbase.id" must appear in
the GROUP BY clause or be used in an aggregate function
```

This Section

- Fetching from a Single Table
- Control Flow
- Aggregating Rows
  - Grouping Rows and Computing Aggregates
  - Filtering on Aggregates
- SQL Rules for Quizzes/Exams

Filtering Groups based on Aggregation

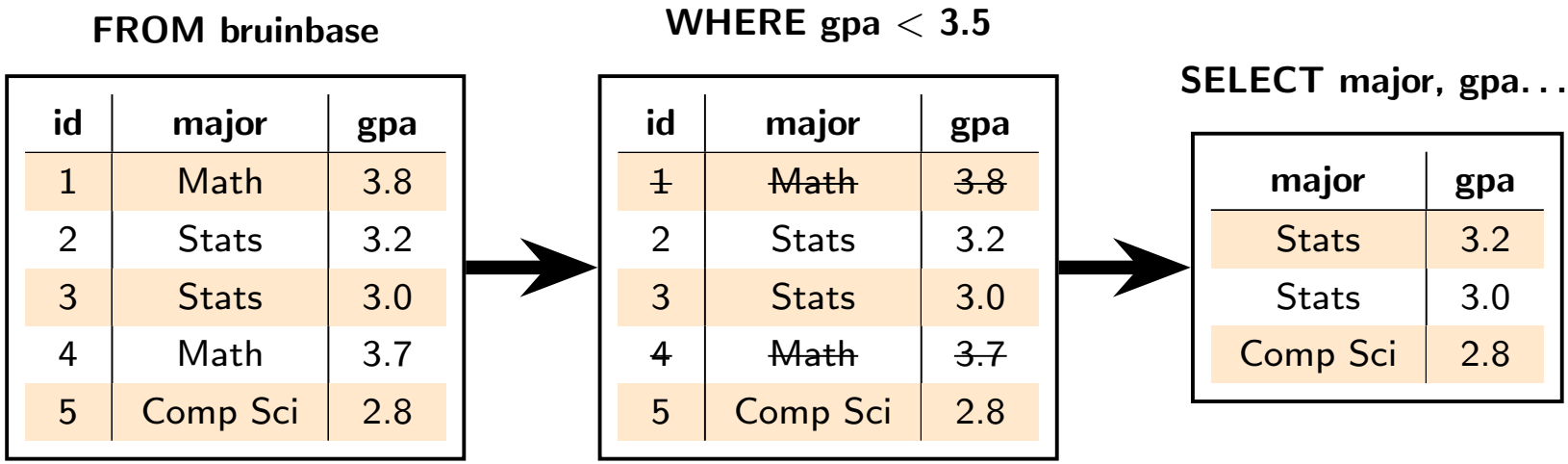
Suppose we want to identify **majors** that have **average** GPAs below 3.5.

We can do that using `HAVING`.

```
SELECT
  major,
  AVG(gpa) AS average
FROM bruinbase
GROUP BY major
HAVING AVG(gpa) < 3.5;
```

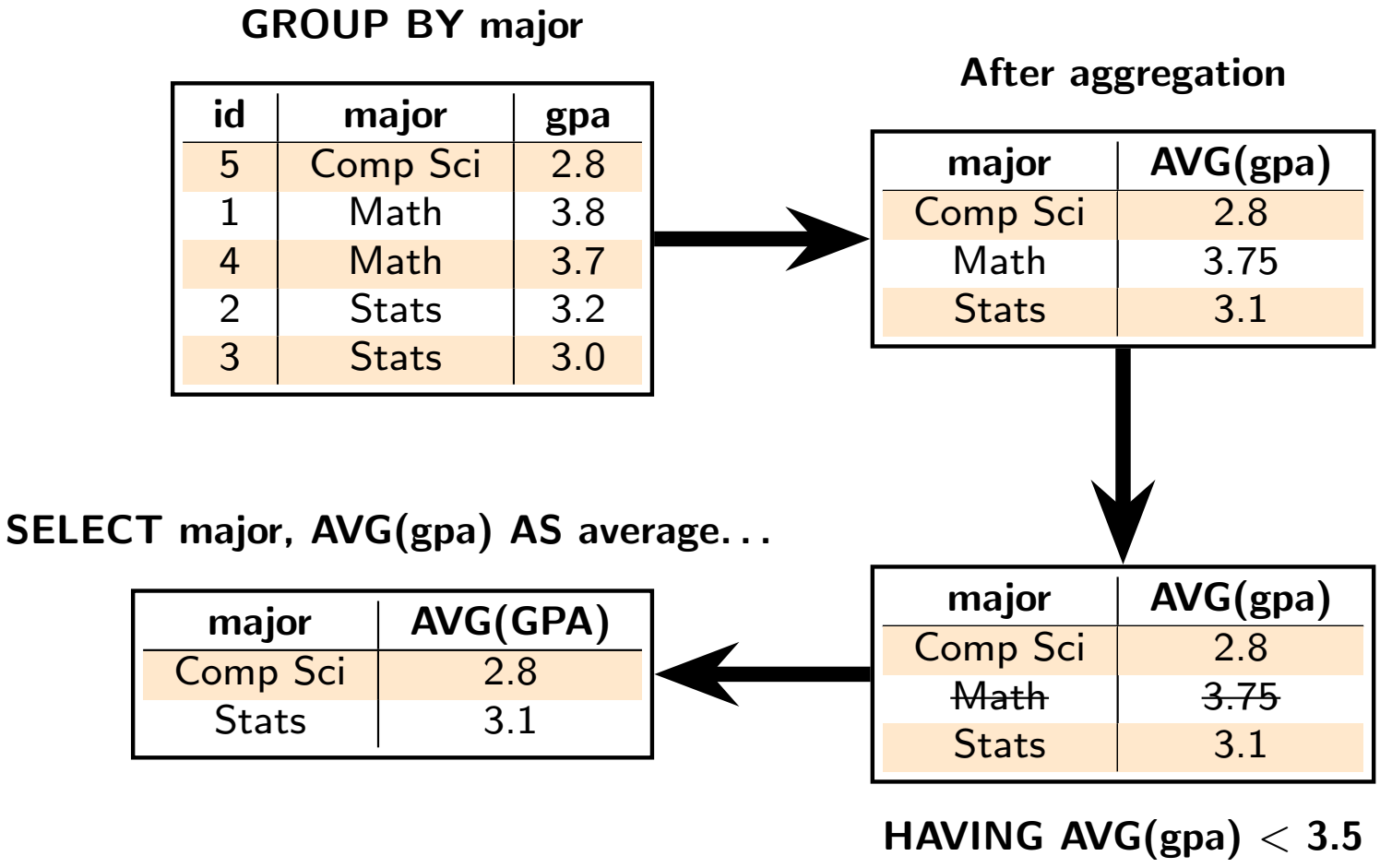
Filtering Groups based on Aggregation

Reminder, `WHERE`: Filter Rows



Filtering Groups based on Aggregation

`HAVING`: Filter Groups using an Aggregation



Exercise

Suppose we do the following analysis: return the undergraduate majors that have an average GPA below 3.5. That is:

1. use only undergraduate GPAs in this calculation.
2. want to compute the average GPA of each major.
3. look at only majors that have an average GPA below 3.5
4. Of those, order them in descending order by average.
5. Display the top 2.

This query involves all of the concepts we just discussed.

Exercise

```sql
SELECT
    major,
    AVG(gpa) AS average
FROM bruinbase
WHERE career = 'UG'
GROUP BY major
HAVING AVG(gpa) < 3.5
ORDER BY average DESC
LIMIT 2;
```

Outline

- Fetching from a Single Table
- Control Flow
- Aggregating Rows
  - Grouping Rows and Computing Aggregates
  - Filtering on Aggregates
- SQL Rules for Quizzes/Exams

Our Evolving List of Rules

1. use full column names in `GROUP BY`, not indexes.
2. do not use column aliases in `HAVING`. PostgreSQL rejects them, and the standard does not allow them.
3. when there is an aggregate function and a `GROUP BY`, everything in the `SELECT` must be in an aggregation function or a `GROUP BY`
4. in `SELECT`, groups come before aggregated columns
5. use `WHERE` for conditions on individual rows and `HAVING` for conditions on groups.

New Order of Statements

When writing a query, the order is as follows, given our new skills:

```
SELECT
FROM
...
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT
```

Order of Execution

Using what we have learned, we can add to the order of execution:

```
FROM
...
WHERE
GROUP BY
HAVING
SELECT
DISTINCT (applies to SELECT list after projection)
ORDER BY
LIMIT
```

Remember that column aliases are created in `SELECT`. Nothing that is executed before it can use the alias. PostgreSQL bends this rule for `GROUP BY` (it accepts an alias there), but please stick to the standard.